

ClinicFlow

Autonomous Patient Follow-Up & Clinic Operations Agent

Hackathon: AWS Agents for Humans

Track: Everyday Agents

Primary Technology: Strands Agents SDK

AWS AI: Amazon Bedrock

Optional/Recommended: Amazon Bedrock AgentCore

Target Users: Small outpatient clinics and their administrative staff

1. Product Summary

One-line pitch

ClinicFlow is an autonomous clinic operations agent that handles patient follow-ups, appointment coordination, reminders, and routine administrative work in the background — and only asks clinic staff to make decisions that genuinely require human judgment.

ClinicFlow is not a medical chatbot.

It is an **autonomous administrative worker** for small clinics.

The system continuously monitors clinic events, identifies tasks that need attention, takes appropriate actions, follows up on unresolved situations, and escalates exceptions to clinic staff.

2. Problem

Small clinics frequently rely on receptionists, nurses, and administrators to manually manage repetitive follow-up work.

Examples:

- Calling patients after consultations
- Sending appointment reminders
- Rescheduling missed appointments
- Following up on pending laboratory results
- Reminding patients about scheduled follow-ups
- Tracking patients who haven't responded
- Updating appointment records
- Notifying staff about unresolved cases

These tasks are individually simple but collectively consume significant staff time.

The real problem

Clinic staff spend valuable time coordinating routine follow-up instead of focusing on patients and higher-value work.

The problem becomes worse when the clinic has dozens of patients requiring different follow-up actions.

3. Product Vision

ClinicFlow should behave like a **virtual clinic operations coordinator**.

A clinic should be able to configure its workflows once and allow ClinicFlow to continuously handle routine administrative follow-up.

Instead of:

```
Staff checks patient list
    ↓
Finds patients requiring follow-up
    ↓
Checks appointment status
    ↓
Contacts patient
    ↓
Waits for response
    ↓
Reschedules manually
    ↓
Updates records
    ↓
Repeats tomorrow
```

ClinicFlow does:

```
Monitor clinic
    ↓
Detect follow-up task
    ↓
Understand patient context
    ↓
Determine safe action
    ↓
Execute action
    ↓
Monitor response
    ↓
```

Follow up automatically



Escalate exceptions

4. Target Users

Primary

Small outpatient clinics

Especially clinics with:

- 1–10 doctors
- Small administrative teams
- 50–500+ active patients
- High appointment volume
- Repetitive follow-up workflows

Secondary

- Clinic administrators
 - Receptionists
 - Practice managers
 - Nurses performing administrative follow-ups
 - Private healthcare practices
-

5. Hackathon Track

Everyday Agents

ClinicFlow fits the Everyday Agents track because the hackathon explicitly includes **health** and emphasizes agents that quietly handle repetitive work in the background.

The project should demonstrate:

Background operation

ClinicFlow doesn't require staff to repeatedly ask:

"Who needs follow-up today?"

It monitors the clinic automatically.

Autonomous action

The agent performs routine administrative tasks itself.

Human escalation

The agent asks humans only when a decision is outside its authority.

This is the central design principle.

6. Important Safety Boundary

ClinicFlow is an **administrative agent**, not a clinical decision-maker.

The agent must never:

- Diagnose patients
- Recommend medication
- Change medication
- Interpret medical results as a clinician
- Determine treatment
- Provide emergency medical advice
- Make clinical decisions

Instead, it can handle:

- Scheduling
- Reminders
- Follow-up coordination
- Administrative communication
- Appointment rescheduling
- Staff notifications
- Workflow tracking

If something appears clinically important, ClinicFlow escalates it to authorized staff.

7. Core User Journey

Step 1 — Clinic Setup

Clinic administrator creates:

- Clinic
- Doctors
- Staff
- Appointment types
- Follow-up rules

- Working hours
 - Communication preferences
-

Step 2 — Patient Consultation Ends

Doctor completes a consultation and creates an administrative follow-up requirement.

Example:

```
Patient: Sarah
Follow-up: 7 days
Appointment required: Yes
Preferred time: Afternoon
```

ClinicFlow receives the event.

Step 3 — Agent Creates Follow-Up Task

The agent determines:

```
Follow-up required:
7 days from consultation

Action:
Contact patient and schedule appointment

Deadline:
September 10
```

The task is added to the agent's workflow.

8. Autonomous Follow-Up

When the follow-up becomes due, ClinicFlow automatically:

1. Checks patient contact information
2. Checks doctor availability
3. Checks clinic availability
4. Determines suitable appointment slots
5. Contacts the patient
6. Proposes available times
7. Processes the response
8. Books the appointment

9. Updates the patient record
10. Logs the completed action

The staff member does not need to manually coordinate the process.

9. Example

Patient Follow-Up

Doctor creates:

Follow-up required:
7 days

Appointment duration:
30 minutes

Preferred doctor:
Dr. Ade

ClinicFlow determines:

Patient follow-up due tomorrow.

It checks:

Dr. Ade availability
↓
Available slots
↓
Patient preferred time
↓
Best matching appointment

The agent sends:

"Your clinic follow-up is due. We have appointments available tomorrow at 2:00 PM and 3:30 PM."

Patient responds:

"2 PM works."

ClinicFlow:

```
Books appointment
↓
Updates patient record
↓
Updates calendar
↓
Sends confirmation
↓
Logs completion
```

No staff intervention required.

10. Missed Appointment Workflow

This should be the **second major demo scenario**.

Patient fails to attend.

ClinicFlow detects:

```
APPOINTMENT_MISSED
```

Agent checks:

- Patient history
- Appointment importance
- Clinic policy
- Available slots

Then sends a rescheduling message.

If patient chooses a valid slot:

```
Agent books new appointment
```

If patient doesn't respond:

```
Agent schedules follow-up reminder
```

If repeated attempts fail:

```
Escalate to clinic staff
```

Example:

Patient follow-up requires staff attention.
Patient has not responded after 3 automated attempts.

This demonstrates persistence.

11. Pending Follow-Up Workflow

ClinicFlow should maintain a queue of unresolved administrative tasks.

Example:

FOLLOW-UP QUEUE

Sarah
Appointment confirmation
Due today
● Waiting for patient

John
Missed appointment
2 reminders sent
△ Staff review recommended

David
Follow-up appointment
Scheduled
✓ Completed

The agent continuously works through the queue.

12. Agentic Workflow

The system should use a genuine agent loop rather than hardcoded if/else automation.

Core loop

```
EVENT
  ↓
OBSERVE
  ↓
RETRIEVE CONTEXT
  ↓
REASON
```



```
↓  
SELECT TOOL  
↓  
EXECUTE ACTION  
↓  
VERIFY RESULT  
↓  
UPDATE STATE  
↓  
MONITOR  
↓  
ESCALATE IF NECESSARY
```

This is the heart of ClinicFlow.

13. Strands Agents SDK

Strands Agents SDK is the **core agent framework**.

Strands supports both Python and TypeScript; for this project, **Python is recommended for the agent backend**, while your existing React/Next.js experience can be used for the frontend. The official Strands quickstart supports custom tools and Amazon Bedrock as the default model provider.

Recommended stack

```
Frontend  
Next.js + TypeScript  
  
Backend  
Python  
  
Agent  
Strands Agents SDK  
  
Model  
Amazon Bedrock  
  
Database  
DynamoDB  
  
Events  
EventBridge  
  
Files  
S3  
  
Deployment  
AgentCore Runtime
```

14. ClinicFlow Agent

Agent name

ClinicFlow Operations Agent

System responsibility

The agent's job is to:

Monitor clinic administrative events, determine what action is required, perform authorized administrative actions using available tools, verify outcomes, maintain follow-up tasks, and escalate situations requiring clinical, financial, safety, or staff judgment.

The agent should understand:

- Current clinic state
- Patient administrative context
- Appointment availability
- Follow-up requirements
- Communication history
- Clinic policies
- Its own authority boundaries

15. Agent Tools

The agent should have explicit tools.

Patient Tools

```
get_patient()  
get_patient_followups()  
get_patient_appointments()  
update_patient_followup()
```

Appointment Tools

```
get_available_slots()  
book_appointment()
```

```
reschedule_appointment()  
cancel_appointment()  
get_appointment()
```

Communication Tools

```
send_patient_message()  
send_staff_notification()
```

Workflow Tools

```
create_followup_task()  
get_pending_tasks()  
complete_task()  
escalate_task()
```

Clinic Tools

```
get_clinic_hours()  
get_doctor_schedule()  
get_clinic_policy()
```

The LLM decides **which tools to use and in what sequence**.

16. Example Agent Decision

Event

```
APPOINTMENT_MISSED
```

The agent receives the event.

It determines:

1. Identify patient
2. Retrieve appointment
3. Check follow-up requirements
4. Check clinic rescheduling policy
5. Find available appointment slots
6. Contact patient
7. Wait for response
8. Process response

- 9. Book appointment
 - 10. Verify booking

This is materially different from a simple chatbot.

17. Human-in-the-Loop

ClinicFlow must have clear boundaries.

Autonomous

The agent can:

- Send appointment reminders
- Send follow-up messages
- Find available appointment slots
- Book routine appointments
- Reschedule according to clinic policy
- Create follow-up tasks
- Send routine staff notifications

Human Approval

Require staff approval for:

- Exceptional scheduling changes
- Fee-related decisions
- Repeated failed contact attempts
- Unusual patient requests
- Exceptions to clinic policy

Human Only

Immediately escalate:

- Medical questions
 - Symptoms
 - Medication questions
 - Lab-result interpretation
 - Emergency situations
 - Clinical concerns
-

18. Human Escalation Example

Patient replies:

"I have been experiencing severe chest pain since yesterday. Should I still come for my appointment?"

ClinicFlow should **not answer the medical question**.

Instead:

⚠ CLINICAL REVIEW REQUIRED

Patient:
Sarah

Message:
Patient reports severe chest pain.

Action:
Escalated to clinic staff.

Automated medical response:
Not provided.

The agent has correctly recognized that the request exceeds its authority.

This is an important demonstration of responsible autonomy.

19. Background Execution

The application should demonstrate that the agent can operate without continuous user prompting.

Event-driven architecture

```
Patient / Clinic Event
  ↓
Amazon EventBridge
  ↓
ClinicFlow Agent
  ↓
Strands Agent
  ↓
Tools
  ↓
AWS Data
  ↓
Action
  ↓
Verification
```

Example EventBridge events:

```
FOLLOWUP_DUE
APPOINTMENT_UPCOMING
APPOINTMENT_MISSED
PATIENT_RESPONDED
FOLLOWUP_OVERDUE
STAFF_ESCALATION_REQUIRED
```

20. AWS Services

Amazon Bedrock

Provides the foundation model used by the Strands agent.

The Strands SDK supports Amazon Bedrock as its default model provider.

Amazon DynamoDB

Primary application database.

Stores:

- Patients
 - Appointments
 - Doctors
 - Follow-ups
 - Tasks
 - Messages
 - Agent actions
 - Escalations
-

Amazon EventBridge

Provides event-driven automation.

Example:

```
FOLLOWUP_DUE
  ↓
EventBridge
```

↓
ClinicFlow

This supports the "background agent" concept.

Amazon S3

Stores:

- Clinic documents
- Policies
- Call sheets/forms
- Optional patient administrative documents

Avoid storing unnecessary sensitive medical information for the hackathon MVP.

Amazon CloudWatch

Used for:

- Logs
 - Errors
 - Agent execution monitoring
 - Operational metrics
-

21. Amazon Bedrock AgentCore

AgentCore is **not required by the hackathon**, but the official rules explicitly say it is a smart architectural choice and can strengthen Technical Implementation.

For ClinicFlow, use:

AgentCore Runtime

Deploy the Strands agent as a managed runtime.

The current AgentCore documentation supports deploying code-based Strands agents to AgentCore Runtime.

AgentCore Memory — Optional

Could be used to maintain useful long-term context such as:

- Patient communication preferences
- Previous administrative interactions

- Clinic workflow preferences

AgentCore provides memory capabilities that can be integrated with Strands.

Recommended priority

Strands Agent	REQUIRED
Bedrock	REQUIRED
DynamoDB	REQUIRED
EventBridge	HIGH
AgentCore Runtime	HIGH
AgentCore Memory	OPTIONAL

Don't let AgentCore complexity delay the actual agent.

22. Frontend

Use your existing strengths here.

Recommended

Next.js + TypeScript + Tailwind

Pages:

```
/dashboard
/patients
/appointments
/followups
/agent
/decisions
/settings
```

23. Dashboard

The dashboard should communicate one thing:

ClinicFlow is working for you.

Example:

```
CLINICFLOW
```


Today's Operations

Patients requiring follow-up	18
Appointments today	32
Automated actions	27
Awaiting patient responses	6
Staff decisions required	2

24. Agent Activity

This should be one of the most prominent screens.

Example:

AGENT ACTIVITY

10:42

Follow-up due for Sarah.

10:42

Retrieved patient follow-up requirements.

10:43

Checking Dr. Ade availability.

10:43

Found 3 available slots.

10:44

Sent appointment options to Sarah.

10:45

Patient selected 2:00 PM.

10:45

Appointment booked.

10:45

Confirmation sent.

✓ Follow-up completed automatically.

This lets judges **see the agent actually doing work.**

25. Human Decisions

Separate automated work from human decisions.

Example:

DECISION REQUIRED

Patient:
John

Issue:
Patient has missed 3 scheduled appointments.

Clinic policy:
Escalate after 3 missed appointments.

ClinicFlow recommendation:
Staff follow-up required.

[REVIEW CASE]

26. Agent Audit Log

Every agent action should be recorded.

Example:

Timestamp
Event
Tool used
Action
Result
Status

Example:

09:31
FOLLOWUP_DUE

09:31
get_patient()

09:31

```
get_available_slots()

09:32
send_patient_message()

09:33
PATIENT_RESPONDED

09:33
book_appointment()

09:33
✓ COMPLETED
```

This makes the system auditable and helps demonstrate technical implementation.

27. Data Model

Patient

```
id
name
contact
preferred_contact_method
communication_preferences
status
```

For the hackathon, use synthetic/demo patient data.

Appointment

```
id
patient_id
doctor_id
date
time
duration
status
type
```

FollowUp

```
id
patient_id
created_at
due_at
type
status
instructions
```

Task

```
id
type
patient_id
priority
status
due_at
assigned_to
```

AgentAction

```
id
event_id
tool
action
status
timestamp
```

Escalation

```
id
patient_id
reason
severity
status
created_at
assigned_to
```

28. MVP Features

Build only these for the hackathon.

Must Have

1. Patient management

Basic synthetic patient records.

2. Appointment management

View, schedule and reschedule appointments.

3. Follow-up engine

Create and track follow-ups.

4. Strands agent

Autonomous operations agent.

5. Agent tools

Patient, appointment, communication and workflow tools.

6. Background events

Event-driven follow-up triggers.

7. Autonomous scheduling

Agent finds and books suitable slots.

8. Missed appointment handling

Agent follows up and attempts rescheduling.

9. Human escalation

Agent escalates exceptional/clinical situations.

10. Agent activity log

Show what the agent actually did.

29. Features NOT Required for MVP

Do not waste hackathon time building:

- Full electronic medical records
- Billing
- Insurance
- Prescription management
- Video consultations
- Patient diagnosis
- Medical image analysis
- Complex hospital integrations
- Native mobile apps
- Real healthcare-provider integrations

The product is about **autonomous administrative work**.

30. Primary Demo Scenario

The demo should tell one coherent story.

Scene 1 — Clinic Starts

Dashboard:

```
18 follow-ups
32 appointments
0 escalations
```

Scene 2 — Follow-Up Becomes Due

System generates:

```
FOLLOWUP_DUE
```

The agent wakes up.

Scene 3 — Agent Works

Show agent activity:

```
Follow-up detected
  ↓
Patient retrieved
  ↓
Doctor availability checked
  ↓
Appointment slots found
  ↓
Patient contacted
```

Scene 4 — Patient Responds

Synthetic patient responds:

"2 PM works."

Agent:

```
Books appointment
  ↓
Updates database
  ↓
Sends confirmation
  ↓
Marks task completed
```

Scene 5 — Missed Appointment

Another patient misses an appointment.

Agent automatically:

```
Detects missed appointment
  ↓
Checks policy
  ↓
Finds available slots
  ↓
Sends rescheduling options
```

Patient chooses a new time.

Agent books it.

Scene 6 — Escalation

Patient sends:

"I'm having severe chest pain. Can I just reschedule?"

Agent recognizes that this is outside its administrative authority.

Instead of answering:

⚠ CLINICAL REVIEW REQUIRED

Case escalated to clinic staff.

This demonstrates **safe autonomy**.

31. Why This Demo Is Strong

One demo demonstrates:

Autonomy

Agent initiates work.

Tool use

Agent interacts with clinic systems.

Reasoning

Agent determines what actions are necessary.

Persistence

Agent waits for and responds to later events.

Background operation

User does not have to repeatedly prompt the agent.

Human-in-the-loop

Agent knows when to stop.

Safety

Agent refuses to cross into clinical decision-making.

That is exactly the behavior we want judges to see.

32. Success Metrics

Track measurable outcomes.

Example

```
Follow-ups processed: 42
Automatically completed: 35
Appointments scheduled: 21
Missed appointments recovered: 8
Staff escalations: 4
Manual interventions avoided: 35
```

Most important metric:

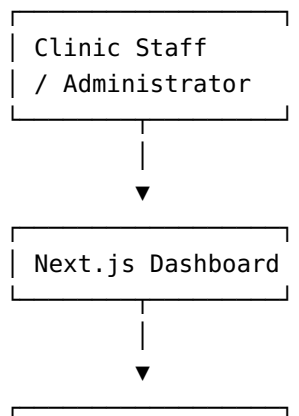
Percentage of routine administrative follow-ups completed without staff intervention.

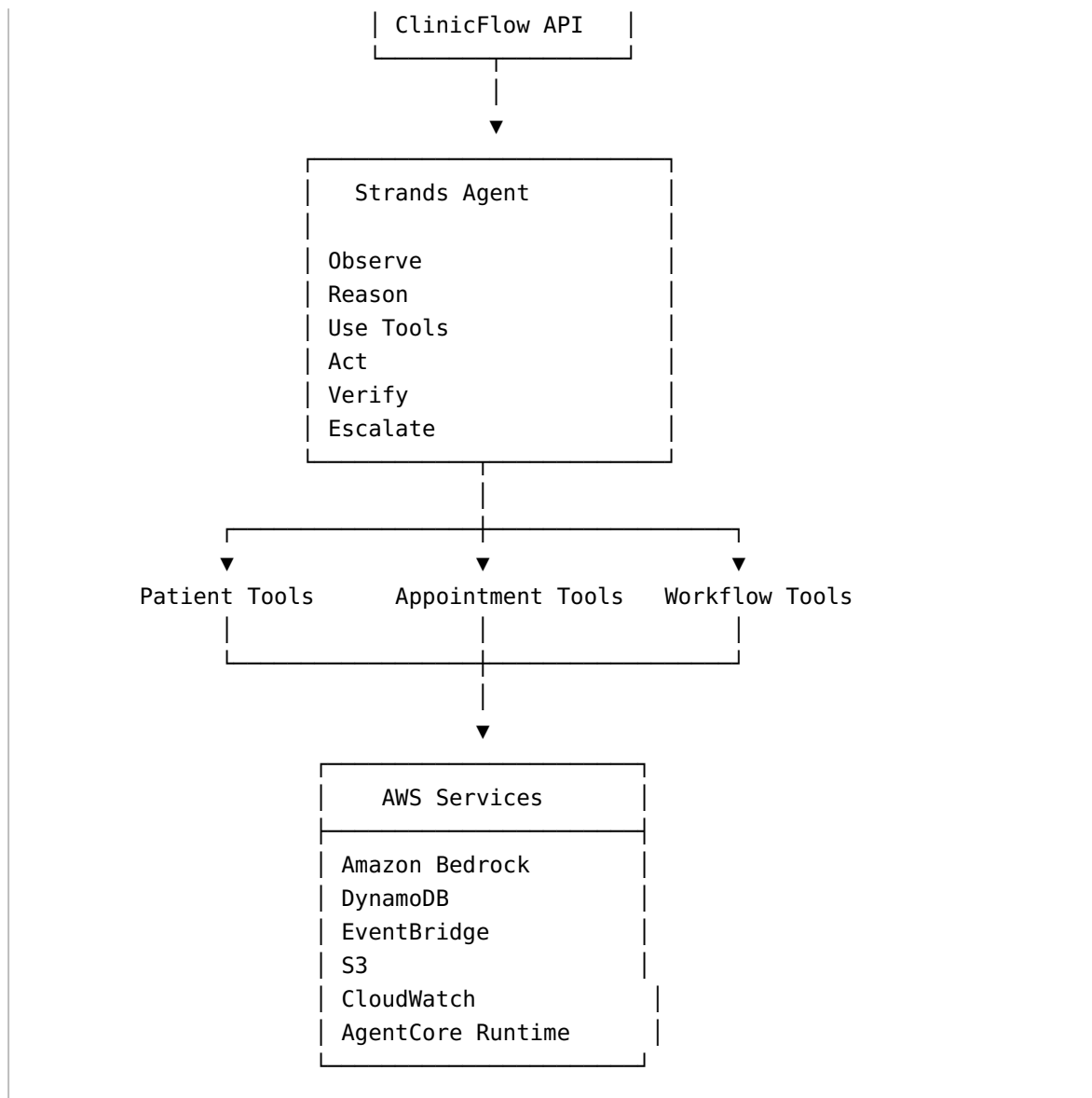
Example:

83% automated resolution

This makes the value proposition concrete.

33. Technical Architecture





34. Build Strategy

Phase 1 — Agent Foundation

Build:

```
Strands
↓
Bedrock
↓
Simple tool
↓
Agent response
```

Get this working first.

Strands is designed to get a basic agent running quickly, and its official examples cover custom tools and deployment patterns.

Phase 2 — Clinic Data

Build:

```
Patient
Appointment
Doctor
FollowUp
Task
```

Use DynamoDB.

Phase 3 — Agent Tools

Implement:

```
get_patient()
get_available_slots()
book_appointment()
reschedule_appointment()
send_patient_message()
create_followup_task()
escalate_task()
```

Phase 4 — Event System

Implement:

```
FOLLOWUP_DUE
APPOINTMENT_MISSED
PATIENT_RESPONDED
FOLLOWUP_OVERDUE
```

Use EventBridge.

Phase 5 — Autonomous Loop

Connect:

```
EventBridge
  ↓
Strands Agent
  ↓
Tools
  ↓
Database
  ↓
Action
  ↓
Verification
```

Phase 6 — Human Escalation

Implement:

```
Agent
  ↓
Detects exception
  ↓
Creates escalation
  ↓
Dashboard notification
  ↓
Staff decision
  ↓
Agent continues
```

Phase 7 — AgentCore

Deploy the Strands agent to AgentCore Runtime.

The current AgentCore CLI supports creating and deploying code-based Strands agents, and AgentCore Runtime is designed for long-running/scalable agent execution.

Phase 8 — Dashboard

Build the UI around the agent's activity.

Do not build the dashboard first.

35. Hackathon Submission Strategy

The official submission requires:

- Public source repository
- MIT/Apache license
- README
- Architecture diagram
- Maximum 5-minute public demo video
- Problem explanation
- Target user
- Why the problem matters
- AWS Builder ID
- Optional live demo

These are explicit submission requirements in the current rules.

README structure

```
# ClinicFlow

## Problem

## Solution

## How the Agent Works

## Architecture

## AWS Services

## Strands Agent

## Agent Tools

## Human-in-the-Loop

## Setup

## Environment Variables

## Running Locally

## Deployment

## Demo
```

Security

License

36. Demo Video

Maximum:

5 minutes

Recommended:

0:00-0:30

Problem.

"Small clinics spend hours every week manually following up with patients and managing appointments."

0:30-0:55

Introduce ClinicFlow.

"ClinicFlow is an autonomous administrative agent that handles those workflows in the background."

0:55-2:00

Show follow-up automation.

2:00-3:00

Show missed appointment → autonomous rescheduling.

3:00-4:00

Show clinical message → safe escalation.

4:00-4:40

Show Strands + AWS architecture and agent activity.

4:40-5:00

Impact.

"ClinicFlow turns repetitive clinic coordination into background work, while keeping important decisions with humans."

37. Competitive Differentiator

Do not pitch:

"AI chatbot for clinics."

Pitch:

"An autonomous administrative worker for clinics."

The difference:

Traditional AI chatbot	ClinicFlow
Waits for user	Runs in background
Answers questions	Performs workflows
One conversation	Persistent tasks
Suggests actions	Executes authorized actions
Human monitors everything	Agent monitors
Generic responses	Clinic-specific policies
No escalation model	Explicit human escalation

38. Core Product Principle

ClinicFlow should follow:

Automate the routine. Escalate the exceptional. Never automate clinical judgment.

This should appear throughout the product, README and demo.

39. Final Product Positioning

Name

ClinicFlow

Category

Autonomous Healthcare Administration Agent

Track

Everyday Agents

Target

Small outpatient clinics

Problem

Repetitive patient follow-up and appointment coordination

Solution

An autonomous agent that monitors, coordinates, executes and follows up on routine administrative workflows.

Technology

Strands Agents SDK + Amazon Bedrock + AWS + AgentCore

Differentiator

Persistent background autonomy with controlled human escalation.

Tagline

ClinicFlow — The clinic's administrative work that happens in the background.